

Gol Language Specification

Version 1.0 — A Go-flavored teaching language for Compiler Design

1. Overview

Gol is a statically-typed, procedural language inspired by Go. It is intentionally minimal, making it suitable as a full-semester compiler project. It supports two primitive types, global and local variables, functions with by-value and by-reference parameters, standard control flow, and a built-in I/O statement.

2. Lexical Specification

2.1 Character Set

Source files are ASCII text. Whitespace (spaces, tabs, newlines) is ignored except as a token separator.

2.2 Comments

```
// this is a single-line comment
/* this is a
   multi-line comment */
```

2.3 Keywords

```
var  func  return  if  else  for
true false  int    bool ref  print println
```

2.4 Operators and Punctuation

+	-	*	/	%	Arithmetic	
==	!=	<	>	<=	>=	Relational
&&		!	Boolean			
=	:=	Assignment / Short declaration				
&	Address-of (for by-ref call sites)					
(	)	{	}	,	;	Punctuation

2.5 Literals

Kind	Pattern	Examples
Integer	<code>-?[0-9]+</code>	<code>0</code> , <code>42</code> , <code>-7</code>
Boolean	<code>true</code> <code>false</code>	<code>true</code> , <code>false</code>
String	<code>" any-char* "</code> (print only)	<code>"hello, world\n"</code>

String literals support escape sequences: `\n` , `\t` , `\\` , `\"` .

2.6 Identifiers

IDENTIFIER $\rightarrow$ [a-zA-Z_][a-zA-Z0-9_]*

Identifiers are case-sensitive. Keywords cannot be used as identifiers.

3. Grammar (EBNF)

$\{ X \}$ = zero or more X

$[X]$ = optional X

$X \mid Y$ = X or Y

Terminals are in "quotes" or ALL_CAPS

program = { topDecl } EOF ;

topDecl = varDecl | funcDecl ;

(* — Declarations — *)

varDecl = "var" IDENTIFIER type ["=" expr] ";" ;

type = "int" | "bool" ;

funcDecl = "func" IDENTIFIER "(" [paramList] ")" [type] block ;

paramList = param { "," param } ;

param = ["ref"] IDENTIFIER type ;

(* — Statements — *)

block = "{" { stmt } "}" ;

stmt = varDecl
| shortDecl
| assignStmt
| ifStmt
| forStmt
| returnStmt
| callStmt
| printStmt ;

shortDecl = IDENTIFIER ":@" expr ";" ; (* local scope only *)

assignStmt = IDENTIFIER "=" expr ";" ;

ifStmt = "if" expr block
{ "else" "if" expr block }
["else" block] ;

forStmt = "for" expr block ; (* while-style *)

returnStmt = "return" [expr] ";" ;

```

callStmt    = IDENTIFIER "(" [ argList ] ")" ";" ;

printStmt   = ( "print" | "println" ) "(" printArg { "," printArg } ")" ";" ;

printArg    = expr | STRING_LIT ;

argList     = arg { "," arg } ;
arg         = [ "&" ] expr ;                      (* & = pass by reference *)

(* — Expressions — *)

expr        = orExpr ;

orExpr      = andExpr { "||" andExpr } ;

andExpr     = notExpr { "&&" notExpr } ;

notExpr     = "!" notExpr | relExpr ;

relExpr     = addExpr [ relOp addExpr ] ;
relOp      = "==" | "!=" | "<" | ">" | "<=" | ">=" ;

addExpr     = mulExpr { ( "+" | "-" ) mulExpr } ;

mulExpr     = unaryExpr { ( "*" | "/" | "%" ) unaryExpr } ;

unaryExpr   = "-" unaryExpr | primary ;

primary     = INT_LIT
            | "true"
            | "false"
            | IDENTIFIER
            | callExpr
            | "(" expr ")" ;

callExpr    = IDENTIFIER "(" [ argList ] ")" ;

```

4. Type System

Gol is **statically typed** with two primitive types:

Type	Size	Range	Zero value
int	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0
bool	1 bit	true or false	false

4.1 Type Rules

- Arithmetic operators (`+` , `-` , `*` , `/` , `%`) require both operands to be `int` ; result is `int` .

- Unary minus (`-`) requires an `int` operand; result is `int` .
 - Relational operators (`==` , `!=` , `<` , `>` , `<=` , `>=`) require both operands to be the **same type**; result is `bool` .
 - `<` , `>` , `<=` , `>=` are only defined for `int` .
 - `==` , `!=` are defined for both `int` and `bool` .
 - Boolean operators (`&&` , `||` , `!`) require `bool` operands; result is `bool` .
 - The condition of `if` and `for` must be `bool` .
 - There is no implicit type conversion.
-

5. Scope Rules

- **Global scope:** `var` declarations at the top level. Visible to all functions.
 - **Function scope:** Parameters and `var` / `:=` declarations inside a function body.
 - **Block scope:** Declarations inside `if` or `for` blocks are local to that block.
 - Shadowing is allowed: a local name hides an outer name of the same identifier.
 - `:=` (short declaration) is only legal inside a function body. It declares **and** initializes.
 - A variable must be declared before use.
-

6. Functions

```
func add(a int, b int) int {
    return a + b ;
}
```

6.1 By-Value Parameters

The default. The function receives a copy; changes do not affect the caller.

```
func double(x int) int {
    x = x * 2 ;
    return x ;
}
```

6.2 By-Reference Parameters (`ref`)

Declare with `ref` ; the caller must pass the address with `&` . Changes to the parameter inside the function are visible at the call site.

```
func increment(ref n int) {
    n = n + 1 ;
}

func main() {
    var counter int = 0 ;
    increment(&counter) ;           // counter is now 1
    println("counter:", counter) ;
}
```

6.3 Return Values

A function may return at most one value. `return;` (no expression) is valid for void functions. A non-void function must have a `return` on every execution path (enforced by semantic analysis).

6.4 Entry Point

The program starts at `func main()`, which takes no parameters and returns no value.

7. Statements

7.1 Variable Declaration

```
var x int ;           // zero-initialized
var ok bool = true;
```

7.2 Short Declaration (local only)

```
result := add(3, 4);
```

The type is inferred from the right-hand side expression.

7.3 Assignment

```
x = x + 1 ;
```

7.4 if Statement

```
if x > 0 {
    println("positive") ;
} else if x < 0 {
    println("negative") ;
} else {
    println("zero") ;
}
```

No parentheses around the condition (Go style). Braces are mandatory.

7.5 for Statement (while-style)

Go uses `for` for all loop forms. Gol adopts the condition-only form:

```
for x < 10 {
    x = x + 1 ;
}
```

No parentheses. Braces mandatory. Equivalent to `while (x < 10)` in C.

7.6 print / println

Built-in output. Accepts any mix of `int` expressions, `bool` expressions, and string literals. `println` appends a newline; `print` does not.

```
println("result:", add(2, 3)) ;
print("x =", x, "\n") ;
```

8. Operator Precedence (high to low)

Level	Operators	Associativity
7	! unary -	Right
6	* / %	Left
5	+ -	Left
4	< > <= >= == !=	Left
3	&&	Left
2	`	
1	(expression)	

9. Complete Example Programs

9.1 Fibonacci (iterative)

```
func fib(n int) int {
    var a int = 0 ;
    var b int = 1 ;
    var i int = 0 ;
    for i < n {
        var tmp int = a + b ; // tmp := a + b (equivalent)
        a = b ;
        b = tmp ;
        i = i + 1 ;
    }
    return a ;
}

func main() {
    var i int = 0 ;
    for i <= 10 {
        println("fib(", i, ") =", fib(i)) ;
        i = i + 1 ;
    }
}
```

9.2 Swap using by-reference

```
func swap(ref a int, ref b int) {
    var tmp int = a ;
    a = b ;
    b = tmp ;
}

func main() {
    var x int = 10 ;
    var y int = 20 ;
    println("before:", x, y) ;
    swap(&x, &y) ;
    println("after:", x, y) ;
}
```

9.3 Boolean logic

```
func isEven(n int) bool {
    return n % 2 == 0 ;
}

func main() {
    var n int = 42 ;
    if isEven(n) && n > 0 {
        println("positive and even") ;
    } else {
        println("odd or non-positive") ;
    }
}
```

9.4 Global variable

```
var maxIter int = 100 ;

func clamp(ref x int) {
    if x > maxIter {
        x = maxIter ;
    }
}

func main() {
    var val int = 200 ;
    clamp(&val) ;
    println("clamped:", val) ;    // prints: clamped: 100
}
```

10. Suggested Compiler Phases

Phase	Output
Lexer	Token stream
Parser	Abstract Syntax Tree (AST)
Semantic Analysis	Decorated AST + Symbol Table
Interpretation	Program Output

11. Suggested Extensions (bonus features)

Feature	Notes
for with init/post	for <code>i := 0; i < n; i = i + 1 { }</code> — classic C-style
break / continue	Loop control; requires a label stack in code gen
Nested functions	Closures are hard; simple nesting without capture is doable
Arrays	var <code>a [10]int</code> — adds index expressions and bounds checking
Multiple return vals	func <code>minmax(a,b int) (int, int)</code> — Go flavor
Type casting	<code>int(expr) / bool(expr)</code> — explicit, no implicit coercion
<code>assert(expr)</code>	Useful for testing; halt with message if false

12. Error Conditions (Semantic Checks)

Your semantic analysis phase must detect and report:

1. **Undeclared variable** — use before declaration.
2. **Redeclared variable** — same name twice in the same scope.
3. **Type mismatch** — e.g., `int + bool`, assigning `bool` to `int`.
4. **Wrong argument count** — call with too few/many arguments.
5. **Argument type mismatch** — passing `bool` where `int` expected.
6. **By-ref without &** — calling a `ref` parameter without the address operator.
7. **& on non-variable** — e.g., `&(x + 1)` is illegal.
8. **Missing return** — non-void function has a code path with no `return`.
9. **return type mismatch** — returning `int` from a `bool` function.
10. **main missing** — no entry point defined.
11. **:= at global scope** — short declaration is only legal locally.